

DEVELOPER DOCUMENTATION

ERGALYON

Invoice Manager — Technical Deep-Dive

Πλήρης τεχνική τεκμηρίωση για CTO / Development Team.
Αρχιτεκτονική, pipelines, algorithms, database schema.

v1.0 — Ιούνιος 2026 — ~21.000 γραμμές κώδικα

Πίνακας Περιεχομένων

- Αρχιτεκτονική Συστήματος & Technology Stack
- Invoice Processing Pipeline (571 LOC)
- Parser System — 3 Επίπεδα (3.431 LOC)
- LLM Orchestrator — Τριπλό AI (225 LOC)
- Scraping Engine — 5 Στρατηγικές (2.976 LOC)
- Catalog Import & AI Column Mapping
- Pylon ERP Export
- NAS Integration & File Watcher
- Database Schema (SQLite + PostgreSQL)
- Authentication & Security
- Στατιστικά Κώδικα & File Map

1. Αρχιτεκτονική Συστήματος

FRONTEND — Next.js 14 (App Router)				
16 pages	shadcn/ui	Tailwind CSS	Framer Motion	
33 API ROUTES (Route Handlers)				
/api/invoices/*	/api/products/*	/api/scraping/*		
/api/suppliers/*	/api/pylon/*	/api/parsers/*		
/api/enrichment/*	/api/catalog/*	/api/chat/*		
SQLite	PgSQL	Ollama	Abacus AI	FlareSolverr
(business)	(auth)	(local AI)	(cloud AI)	(CF bypass)
better-sqlite3	Prisma ORM	8B + 70B Llama3.1	Claude Sonnet	Docker container

Technology Stack

Layer	Technology	Details
Frontend	Next.js 14, React 18	App Router, Tailwind CSS, shadcn/ui, Framer Motion, Lucide Icons
Backend	Next.js API Routes	33 route handlers, TypeScript, Route-level auth guards
Business DB	SQLite (better-sqlite3)	8 tables: products, invoices, invoice_items, suppliers, enrichment_queue, scrape_configs, parser_configs, product_images
Auth DB	PostgreSQL (Prisma)	1 table: users (NextAuth.js credentials provider)
AI — Tier 1	Ollama Llama 3.1 8B	Chat, column mapping — local, zero cost, 60s timeout
AI — Tier 2	Ollama Llama 3.1 70B	Analysis, parser generation — local, zero cost, 120s timeout

AI — Tier 3	Abacus AI Claude Sonnet	Invoice extraction, vision — cloud, pay-per-token, 180s timeout
Scraping	Custom 5-level engine	WC-API → HTTP+JSON-LD → FlareSolvrr → Google → Playwright+AI Vision
OCR	Tesseract + pdftotext	Greek+English (ell+eng), 300 DPI, PSM 6
Server	Hetzner Dedicated	176.9.105.236:3004, Docker for FlareSolvrr

2. Invoice Processing Pipeline

app/api/invoices/process/route.ts — 571 γραμμές

POST /api/invoices/process — FormData { file, supplier_id? }

2.1 Text Extraction — Τριπλή Στρατηγική

A. Κανονικό PDF (text-based)

```
// pdftotext με -layout flag για διατήρηση δομής στηλών
execSync('pdftotext -layout "${filePath}" "${txtPath}"')
// Αν rawText.trim().length > 50 → επιτυχία
```

B. Σκαναρισμένο PDF (OCR)

```
// Μόνο αν rawText < 50 χαρακτήρες (= σκαναρισμένο)
execSync('pdftoppm -r 300 -png "${filePath}" "${imgPrefix}"')
// Για κάθε σελίδα-εικόνα:
execSync('tesseract "${imgFile}" "${outBase}" -l ell+eng --psm 6')
// Συνένωση σελίδων σε ένα κείμενο
```

Γ. Excel/XLSX

```
const workbook = XLSX.read(buffer, { type: 'buffer' })
const data = XLSX.utils.sheet_to_json(sheet, { header: 1 })
// Κάθε γραμμή → tab-separated → join('\n')
```

2.2 Parser Selection — Ιεραρχία 3 Επιπέδων

Ποή: Hardcoded TS Parsers (12) → Dynamic JSON Parsers (DB) → LLM Extraction (AI fallback)

tryDirectParse(text, supplierId?) στο lib/parsers/index.ts (230 LOC):

- Αν `supplierId` → δοκιμάζει **μόνο** τον parser αυτού του προμηθευτή
- Αν δεν δοθεί → loop σε **ΟΛΟΥΣ** τους parsers σειριακά
- Κάθε parser: `canParse(text): boolean` → regex patterns
- Αν `canParse() === true` → `parse(text): ParsedInvoice`

2.3 LLM Extraction (Fallback)

Αν κανένας parser δεν ταίριαζει:

```
// 1. Compact κείμενο αν > 4000 chars
function compactInvoiceText(text): string {
  // Αφαιρεί κενές γραμμές, πολλαπλά spaces
  // Κόβει header/footer boilerplate
  // Κρατάει μόνο γραμμές με αριθμούς (items)
}

// 2. TEXT MODE → LLM (Tier 3 Abacus Claude)
prompt = 'Extract invoice items... Return JSON:
{ supplier_name, supplier_afm, items: [...] }'
response = await callLLM(prompt, 'invoice-extraction')
```

```
// 3. Αν TEXT αποτύχει FILE MODE
// Base64 encode PDF → file attachment στο Claude
```

2.4 Product Matching — 4 Επίπεδα

Επίπεδο	Μέθοδος	Confidence	Λογική
1	BARCODE	1.0	WHERE barcode = ?
2	EXACT_CODE	1.0	WHERE supplier_code = ? OR factory_code = ?
3	NORMALIZED	0.95	Strip spaces/dots/dashes → compare uppercase
4	PARTIAL	0.7	code.includes(searchCode) ή αντίστροφα

2.5 Database Update

```
if (match) {
  // UPDATE: cost_price, last_invoice_date, factory_code (COALESCE)
  db.prepare('UPDATE products SET cost_price=?,
    factory_code=COALESCE(NULLIF(factory_code,\'\''),?)')
} else {
  // INSERT: source='AUTO_ADDED', status='pending_review'
  // → enrichment_queue (missing_fields, priority)
}
```

3. Parser System — 3 Επίπεδα

Σύνολο: 3.431 γραμμές κώδικα σε 13 αρχεία

3.1 Hardcoded TypeScript Parsers (12 αρχεία)

Κάθε parser υλοποιεί `canParse(text): boolean` + `parse(text): ParsedInvoice`

Παράδειγμα: Makita Parser (220 LOC)

```
canParse(text: string): boolean {
  return /MAKITA|MAKITA/i.test(text) &&
    /ΤΙΜΟΛΟΓΙΟ|INVOICE|ΔΕΛΤΙΟ/i.test(text)
}

parse(text: string): ParsedInvoice {
  // 1. Extract supplier info: AΦM regex /A\.\?Φ\.\?M\.\?s*:\?s*(\d{9})/
  // 2. Split σε lines
  // 3. Per line regex extract:
  //    - product code: /^(w{2,3}\d{4,})/
  //    - description: middle portion
  //    - quantity: /(\d+[\.,]?\d*)s*(TEM|PCS)/
  //    - unit_price, discount columns, net_price
  // 4. Calculate: net = price * (1-disc1/100) * (1-disc2/100)
}
```

Parser	Αρχείο	LOC	Detection Pattern
Makita	makita.ts	220	MAKITA MAKITA + ΤΙΜΟΛΟΓΙΟ
DeWalt/Stanley/B&D/Irwin/Lenox	dewalt.ts	350+	AΦM + brand names
Bosch	bosch.ts	280	BOSCH + invoice patterns
HiKoki	hikoki.ts	200	HIKOKI HITACHI
Milwaukee	milwaukee.ts	250	MILWAUKEE + ΤΙΜΟΛΟΓΙΟ
Metabo	metabo.ts	220	METABO + AΦM
Vector Brands	vectorbrands.ts	200	Phone: 21210210210
+ 5 ακόμα	*.ts	~900	Various patterns

3.2 Dynamic JSON Parser Engine

lib/parsers/generic.ts — 411 γραμμές

```
// JSON parser config structure (stored in DB)
{
  "supplier_detection": { "patterns": ["REGEX1", "REGEX2"] },
  "line_pattern": "REGEX_WITH_NAMED_GROUPS",
  "field_mappings": {
    "code": { "group": "code" },
    "quantity": { "group": "qty", "transform": "parseGreekNumber" }
  },
  "skip_patterns": ["^ΣΥΝΟΛΟ", "^---"],
  "header_pattern": "ΚΩΔΙΚΟΣ.*ΠΕΡΙΓΡΑΦΗ.*ΠΟΣΟΤΗΤΑ"
}
```

Εκτέλεση: (1) `canParse()` : Όλα τα `supplier_detection.patterns` — ≥ 1 match $\rightarrow$ true. (2) `parse()` : Βρίσκει `header_pattern` $\rightarrow$ parsing μετά το header $\rightarrow$ κάθε γραμμή `line_pattern.exec()` $\rightarrow$ named groups $\rightarrow$ transforms (`parseNumber` , `parseGreekNumber` : αφαιρεί `.` χιλιάδων, αντικαθιστά `,` $\rightarrow$ `.`).

3.3 AI Parser Generation

`app/api/parsers/generate/route.ts` — 206 γραμμές

Ροή: Upload sample PDF $\rightarrow$ pdftotext $\rightarrow$ LLM prompt ("Ανάλυσε, δημιούργησε JSON parser config") $\rightarrow$ validate JSON $\rightarrow$ test against sample $\rightarrow$ save to DB $\rightarrow$ Visual Regex Editor για fine-tuning.

4. LLM Orchestrator — Smart Routing

lib/llm.ts — 225 γραμμές

4.1 Model Routing

Task Type	Tier	Model	Timeout	Cost
chat, column-mapping	1	Ollama Llama 3.1 8B	60s	€0
analysis, parser-generation	2	Ollama Llama 3.1 70B	120s	€0
invoice-extraction, vision	3	Abacus AI Claude Sonnet	180s	per-token

4.2 Fallback Chain

```
callLLM(prompt, complexity, options?) {
  config = getModelConfig(complexity)

  if (config.provider === 'ollama') {
    try {
      response = await fetch('http://localhost:11434/api/generate', {
        body: { model: config.model, prompt, stream: false,
          options: { temperature: 0.1 } },
        signal: AbortSignal.timeout(config.timeout)
      })
      return data.response
    } catch {
      // AUTO FALLBACK → Tier 3 (Abacus Claude)
      return callAbacusAI(prompt, options)
    }
  }
  return callAbacusAI(prompt, options)
}
```

Σημαντικό: Αν `taskType === 'invoice_extraction'`, το σύστημα θέτει `effectiveSkipOllama = true` — παρακάμπτει εντελώς το Ollama (αργό σε CPU-only servers) και πηγαίνει κατευθείαν στο Abacus AI.

5. Scraping Engine — 5 Στρατηγικές

lib/product-scraper.ts (1.620 LOC) + lib/scraper.ts (394 LOC) + lib/browser-scraper.ts (962 LOC)

5.1 Waterfall Strategy

#	Στρατηγική	Πότε	Τεχνολογία
1	WooCommerce REST API	Site με WC-API keys	GET /wp-json/wc/v3/products?search={code}
2	HTTP + JSON-LD	Open sites	fetch(url) → parse JSON-LD / Open Graph meta
3	FlareSolvrr	Cloudflare sites	Docker container → headless Chrome → CF bypass
4	Google Search	Broken site search	google.com/search?q={code}+site:{domain}
5	Playwright + AI Vision	Nuclear option	Screenshot → base64 → Claude vision analysis

5.2 FlareSolvrr — Temp File Pattern

```
// Temp file pattern — αποφυγή shell escaping issues
const tmpFile = '/tmp/flare_' + Date.now() + '.json'
fs.writeFileSync(tmpFile, JSON.stringify({
  cmd: 'request.get',
  url: searchUrl,
  maxTimeout: 60000
}))
execSync('curl -s -X POST http://localhost:8191/v1' +
  ' -H "Content-Type: application/json" -d @"' + tmpFile)
fs.unlinkSync(tmpFile)
```

5.3 Playwright + AI Vision

```
// Webpack bypass trick
const playwright = eval('require')('playwright')

// Ποή:
// 1. browser.newPage() → goto(searchUrl)
// 2. page.screenshot() → base64 PNG
// 3. LLM Tier 3: "Βρες προϊόν {code} σε αυτή τη σελίδα"
// 4. AI αναγνωρίζει → click product link
// 5. Product page screenshot → AI extraction
// 6. Return: { title, price, sku, image_url, availability }
```

5.4 Relevance Filtering

4-βάθμια: Exact SKU match → Normalized comparison → Code in title → Levenshtein distance ≤ 2

6. Catalog Import & AI Column Mapping

app/api/catalogs/upload/route.ts + app/api/catalogs/import/route.ts

6.1 Column Mapping — Dual Strategy

```
// Strategy A: autoMapByName — regex patterns
/κωδ|code|sku/i      → supplier_code
/μπn|factory|εργοστ/i → factory_code
/barcode|ean|gtin/i  → barcode
/περιγρ|desc/i       → description
/τιμ[ηή]|price|κόστ/i → cost_price

// Strategy B: LLM mapping (Tier 1) αν autoMap αποτύχει
prompt = "Map these headers to fields: {headers} → {fields}"
```

6.2 Import Modes

Mode	Λογική
all (default)	UPDATE existing + INSERT new products
existing_only	UPDATE μόνο υπάρχοντα — skip νέα

6.3 Double-Header Detection

Αν row[0] περιέχει generic headers ("Column1", "Στήλη1") → χρησιμοποιεί row[1] ως actual headers.

7. Pylon ERP Export

app/api/pylon/export/route.ts — 196 γραμμές

IMPORT Format (νέα προϊόντα)

SUPPLIER_CODE, FACTORY_CODE, DESCRIPTION, PYLONCODE, COST_PRICE, COST_PRICE_DATE, SELL_PRICE,
DESCRIPTION2, BARCODE, COST_PRICE, SELL_PRICE, SELL_PRICE_DATE
VAT_RATE, CATEGORY, BRAND, UNIT, WEIGHT

UPDATE Format (υπάρχοντα)

Αυτόματη επιλογή: αν product.pylon_code → UPDATE format, αλλιώς → IMPORT format.

8. NAS Integration

lib/nas.ts — 104 γραμμές

```
// Safe path checking — αποτρέπει directory traversal
function safePath(basePath, relativePath): string {
  const resolved = path.resolve(basePath, relativePath)
  if (!resolved.startsWith(basePath))
    throw new Error('Path traversal detected')
  return resolved
}

// Safe NAS check — 3-second timeout αντί fs.existsSync
function safePathExists(p, timeoutSec = 3): boolean {
  execSync('timeout ' + timeoutSec + ' test -d "' + p + '"')
  // Αν NAS unresponsive → timeout → return false
  // Αποτρέπει hang ολόκληρου του server
}
```

// File watcher: 3-sec debounce → auto-process PDF/XLSX

9. Database Schema

9.1 SQLite — Business Data (8 tables)

```
CREATE TABLE products (  
  id INTEGER PRIMARY KEY,  
  supplier_code TEXT,           -- Κωδικός προμηθευτή  
  factory_code TEXT,           -- MPN (manufacturer part number)  
  barcode TEXT,                -- EAN/GTIN  
  description TEXT, description2 TEXT,  
  cost_price REAL,             -- Τιμή κόστους (από τιμολόγιο)  
  sell_price REAL, wholesale_price REAL, retail_price REAL,  
  vat_rate REAL DEFAULT 24,  
  category TEXT, brand TEXT,  
  supplier_id INTEGER,         -- FK → suppliers  
  pylon_code TEXT,             -- Κωδικός Pylon ERP  
  status TEXT DEFAULT 'active', -- active/pending_review/archived  
  source TEXT,                 -- MANUAL/AUTO_ADDED/CATALOG_IMPORT  
  completion_percentage REAL,  
  missing_fields TEXT,         -- JSON array  
  cost_price_updated_at TEXT,  
  created_at TEXT DEFAULT (datetime('now'))  
);  
  
CREATE TABLE invoices (  
  id, supplier_id, invoice_number, invoice_date,  
  total_amount, file_path, status, items_count,  
  matched_count, completion_percentage, processed_at  
);  
  
CREATE TABLE invoice_items (  
  id, invoice_id, product_id,      -- NULL av no match  
  supplier_code, description,  
  quantity, unit_price, discount, net_price,  
  match_method,                   -- BARCODE/EXACT/NORM/PARTIAL/NEW  
  match_confidence                -- 0.7 - 1.0  
);  
  
CREATE TABLE suppliers (id, name, afm, email, phone, parser_type);  
CREATE TABLE enrichment_queue (id, product_id, missing_fields, priority, status);  
CREATE TABLE scrape_configs (id, site_domain, strategy, config_json, enabled);  
CREATE TABLE parser_configs (id, supplier_id, config_json, test_accuracy);  
CREATE TABLE product_images (id, product_id, url, source_site, is_primary);
```

9.2 PostgreSQL — Auth (Prisma ORM)

```
model User {  
  id      String   @id @default(cuid())  
  email   String   @unique  
  password String   // bcrypt hashed  
  name    String?  
  role    String   @default("user")  
  createdAt DateTime @default(now())  
  @@map("users")  
}
```

10. Authentication & Security

Auth Provider	NextAuth.js credentials (email + bcrypt password)
Route Protection	Middleware matcher + getSession(authOptions) σε κάθε API
Path Traversal	safePath() σε NAS operations
SQL Injection	Prepared statements (better-sqlite3) + Prisma parameterized
API Keys	Environment variables μόνο — ποτέ στον κώδικα
NAS Timeout	3-sec safePathExists() — αποτρέπει server hang

11. Στατιστικά Κώδικα & File Map

Κατηγορία	Γραμμές	Αρχεία	Key Files
Frontend (app/)	9.472	49	16 pages, 33 API routes
Business Logic (lib/)	7.063	20	llm.ts, sqlite.ts, nas.ts, auth.ts
UI Components	4.632	~30	navbar, stat-card, status-badge + shadcn/ui
Invoice Parsers	3.431	13	12 hardcoded + generic.ts engine
Scraping Engine	2.976	3	product-scraper, scraper, browser-scraper
ΣΥΝΟΛΟ	~21.000	100+	

API Route Map (33 endpoints)

Κατηγορία	Endpoints
Invoices	/api/invoices, /upload, /process, /[id]
Products	/api/products, /[id], /bulk, /[id]/images, /[id]/scrape
Suppliers	/api/suppliers, /[id], /[id]/parser
Scraping	/api/scraping/run, /configs, /browser-test
Catalogs	/api/catalogs/upload, /import
Parsers	/api/parsers, /generate, /test
Export	/api/pylon/export, /api/export/products
Other	/api/stats, /enrichment, /chat, /nas/status, /database/reset, /auth/*

UI Page Map (16 σελίδες)

Σελίδα	URL	Λειτουργία
Dashboard	/	Stats, πρόσφατα τιμολόγια, danger zone
Invoices	/invoices	Λίστα τιμολογίων, delete
Upload	/invoices/upload	Upload & process, parser creation mode
Results	/invoices/results/[id]	Αποτελέσματα ανά τιμολόγιο
Products	/products	Πλήρης διαχείριση, inline edit, bulk, images
Product Detail	/products/[id]	Λεπτομέρειες & edit
Suppliers	/suppliers	Λίστα, δημιουργία, parser wizard
Supplier Detail	/suppliers/[id]	Στοιχεία, edit, delete
Parser Editor	/suppliers/[id]/parser	Visual regex editor
Catalog Upload	/catalogs/upload	4-step wizard import

Enrichment	/enrichment	Queue management
Scraping	/scraping	Web scraping management
Pylon Export	/pylon	ERP export
NAS Monitor	/nas	NAS status & logs
Chat	/chat	AI RAG assistant
Login	/login	Authentication